

Part 3 – Docker Compose

Docker Compose lets you manage **multiple containers** with a single configuration file.

Instead of running several long `docker run` commands, you describe everything in one file.

`docker-compose.yml`

or, in newer Docker versions, commonly:

`compose.yml`

Then start everything with:

`docker compose up`

Why Use Docker Compose?

Imagine a Node.js application that needs:

- Node.js
- MongoDB
- Redis

Without Compose:

`docker run ...`

`docker run ...`

`docker run ...`

With Compose:

`docker compose up`

Everything starts together.

Basic Structure

services:

app:

image: node:22

db:

image: mongo

Everything lives under the **services** section.

Example Project

project/

- |
- |— Dockerfile
- |— docker-compose.yml
- |— server.js
- |— package.json

Build from Dockerfile

Instead of using an existing image:

services:

app:

build: .

Compose will execute:

`docker build .`

automatically.

Ports

ports:

- "3000:3000"

Equivalent to:

```
docker run -p 3000:3000
```

Environment Variables

environment:

PORT: 3000

NODE_ENV: production

Equivalent to:

```
docker run -e PORT=3000
```

Container Name

container_name: my-api

Instead of a randomly generated name.

depends_on

Controls startup order.

services:

app:

depends_on:

- db

db:

image: mongo

Compose starts **db** before **app**.

Note: **depends_on** only controls startup order. It does **not** wait until the database is fully ready to accept connections.

Networks

Containers in the same Compose project are connected to a default network automatically, so they can communicate using their **service names**.

services:

app:
 image: node

db:
 image: mongo

Inside the **app** container, you can connect to MongoDB using:

mongodb://db:27017

You can also define custom networks:

networks:

 backend:

services:

app:
 networks:
 - backend

db:
 networks:
 - backend

Volumes

Volumes persist data even if containers are removed.

services:

db:
 image: mongo
 volumes:
 - mongo-data:/data/db

volumes:
mongo-data:

The MongoDB data remains available across container recreations.

Complete Example

services:

app:
 build: .
 container_name: node-api

ports:
 - "3000:3000"

environment:
 PORT: 3000

depends_on:
 - db

db:
 image: mongo

 container_name: mongodb

volumes:
 - mongo-data:/data/db

volumes:
mongo-data:

Common Docker Compose Commands

Start services:

docker **compose up**

Start in the background:

docker **compose up -d**

Stop services:

docker **compose down**

Build images:

docker **compose build**

View running services:

docker **compose ps**

View logs:

docker **compose logs**

Follow logs:

docker **compose logs -f**

Restart services:

docker **compose restart**

Overall Workflow

Write Dockerfile



docker build



Docker Image



docker run



Single Container

Multiple Containers



`docker-compose.yml`



`docker compose up`



App + Database + Redis + Other Services